d 次元 Apriori-Window: 多次元トランザクションデータにおける密集アイテムセット領域の厳密抽出

Anonymous Author(s)
Anonymous Institution

ABSTRACT

Apriori-Window は時刻付きトランザクションデータに対してスライディングウィンドウ密集区間を直接かつ厳密に発見する手法であるが、1 次元の時間軸のみを扱うため、空間的に局在するパターンを識別できない問題がある。本稿では Apriori-Window を任意の d 次元グリッド（時間 + $d-1$ 個の空間・カテゴリ次元）に拡張した **d 次元 Apriori-Window アルゴリズム**を提案する。3 つの高速化技術—d 次元接頭辞和（2^d コーナー包除）、t 軸ストライド、Apriori 枝刈り—はいずれも前身手法の直接拡張であり、計算量をナイーブ実装の $O(M \cdot G \cdot \prod W_k)$ から $O\bigl(M \cdot G + N \cdot \prod_{k \geq 1} W_k\bigr)$ に削減する。

合成データ実験（d=3）では、提案手法が植え付けパターン 3 件を Precision = 1.0・Recall = 1.0 で完全回収する一方、1 次元相当設定は 93〜100 件の偽陽性パターンを生成した（Precision ≈ 0.03）。次元数スケーラビリティ実験（d=1〜5）では実行時間の次元依存性を定量化した。N = 100,000 トランザクション・d=3 で約 1.4 秒（Rust + Rayon 並列化）を達成した。

KEYWORDS

頻出パターンマイニング, 多次元データ, スライディングウィンドウ, 密集領域検出, Apriori, d 次元接頭辞和

1 はじめに

頻出パターンマイニング [1, 5] はトランザクションデータ中に繰り返し出現するアイテムセットを発見する基盤技術であり、小売分析、医療データ解析、Web ログ分析など幅広い分野に応用されている。

Apriori-Window [2] は、タイムスタンプ付きトランザクションデータベースに対して、スライディングウィンドウのサポートが閾値 θ 以上となる密集区間とそこで頻出するパターンを効率的に発見するアルゴリズムである。同手法は t 軸ストライド最適化と Apriori 枝刈りを組み合わせることで、候補区間の評価コストを大幅に削減し、LPFIM や LPPM などの既存手法を性能で上回ることを示している。

しかし現実のトランザクションデータは時刻だけでなく**空間的位置情報**（店舗座標、地域コード、GPS 位置等）を伴うことが多い。例えば、小売チェーンでは各トランザクションが特定の店舗（座標）で発生し、疫学データでは症例の発生場所が記録される。このような時空間トランザクションデータにおいて、「沿岸部店舗のみで集中的に売れるアイテムセット」のような**時空間局所的な密集パターン**を発見することは、時間軸のみを扱う Apriori-Window では不可能である。全店舗データを時間軸に集約してしまうと、空間的に局在するパターンが背景ノイズに埋もれてしまうからである。

1.1 技術的課題

Apriori-Window を 1 次元（時間軸）から 3 次元（時間 × 空間 x× 空間 y）に拡張するには以下の課題がある。

(1) **支持曲面の計算爆発**：グリッド $(T \times X \times Y)$ 上の全パターン・全グリッド点でサポートを評価する単純実装は $O(M \cdot G \cdot W_t W_x W_y)$ の計算量を要する。$G = T \times X \times Y$ はグリッドサイズ、M はパターン数、$W = W_t W_x W_y$ はウィンドウ体積である。
(2) **次元増加による候補爆発**：空間次元が加わると候補グリッド点が $T \to T \times X \times Y$ に増大し、各パターンについてすべての点を評価するコストが膨大になる。
(3) **t 軸ストライドの 3 次元化**：Apriori-Window の 1 次元ストライドアルゴリズム [2] を各空間ウィンドウ (x_s, y_s) に独立に適用できるよう構造化する必要がある。
(4) **密集領域定義の一般化**：1 次元の「密集区間」を 3 次元の「密集領域（連結成分）」に自然に拡張し、Apriori 枝刈りを密集セル集合の交差として定式化する必要がある。

1.2 本研究の貢献

本稿では Apriori-Window [2] を時間 × 空間 (x, y) の 3 次元グリッドに拡張した **3 次元 Apriori-Window アルゴリズム**を提案する（4 節）。貢献は以下の 3 点である。

(1) **3 次元接頭辞和による $O(1)$ ウィンドウクエリ**：出現グリッド I_P に対して $(T{+}1) \times (X{+}1) \times (Y{+}1)$ 接頭辞和を構築し、8 コーナー包除原理により任意ウィンドウのサポートを $O(1)$ で取得する。これは 1 次元の累積和クエリを 3 次元に拡張したものである。
(2) **t 軸ストライドの 3 次元拡張**（$k = 1$）：Apriori-Window の t 軸ストライドアルゴリズム [2] を各空間ウィンドウ (x_s, y_s) の XY イベントリストに独立に適用することで、不要な時間スキャンを排除する（補題 2 で正確性を証明）。
(3) **3 次元 Apriori 枝刈り**（$k \geq 2$）：Apriori-Window の候補区間交差枝刈り [2] を 3 次元密集セル集合の交差として拡張する（補題 1）。交差集合が空のパターンは I_P 構築を省略でき、大幅な枝刈りが可能である。

合成データ実験（5 節）では、3 次元設定（$W_x = W_y = 3$）が植え付け 3 パターンを Precision = 1.0、Recall = 1.0 で完全回収する一方、1 次元相当の設定（$W_x = W_y = 50$）では背景ノイズ由来の偽陽性パターンが多数検出される（Precision ≈ 0.02）ことを示し、空間次元の明示的なモデル化の重要性を実証する。また N = 100,000 トランザクションで約 1.5 秒（Rust + Rayon 並列化）を達成し、Apriori 枝刈りが $k \geq 3$ の候補を完全に排除することを確認した。

2 関連研究

本研究は、時空間トランザクションデータから密集アイテムセット領域を発見する問題を扱う。この問題は複数の研究領域にまたがるため、各領域の代表的手法と本研究の相違点を整理する。

2.1 頻出パターンマイニングと Apriori アルゴリズム

Agrawal ら [1] の Apriori アルゴリズムは、アイテムセットの単調性（部分集合は常に親集合以上のサポートを持つ）を利用した枝刈りにより、候補数を大幅に削減する。Han ら [5] の FP-Growth は接頭辞木を用いてさらに高速化する。これらは静的なトランザクションデータベースを対象とし、時間的・空間的なパターン分布の変化は扱わない。

本研究は Apriori の枝刈り原理を 3 次元密集セル集合に拡張し、候補セルの交差集合が空の場合に I_P 構築を省略する（§4.3）。

2.2 Apriori-Window（直接の前身研究）

Apriori-Window [2] は、タイムスタンプ付きトランザクションデータに対してスライディングウィンドウサポートが閾値を超える密集区間を直接かつ厳密に発見するアルゴリズムである。t 軸ストライドにより非密集区間を効率的にスキップし、Apriori 枝刈りにより $(k-1)$-部分集合に密集区間がないパターンを早期除去する。比較実験では LPFIM [5]、LPPM、Recur を大幅に上回る性能を示した。

本研究はこの手法を 1 次元（時間軸のみ）から 3 次元（時間 × 空間 x× 空間 y）に拡張する。$W_x = W_y = 1$ とすると本研究の問題設定・アルゴリズムは Apriori-Window に帰着する（注意 1、補題 2）。

2.3 スライディングウィンドウパターンマイニング

スライディングウィンドウを用いて時間的に変化するパターンを追跡する研究として、Teng ら [3] は変化点ベースのイベント帰属パイプラインを提案している。このパイプラインは 1 次元の時間軸のみを扱い、空間的に局在するパターン変動は検出できない。

2.4 時空間パターンマイニング

軌跡パターンマイニング [4] は移動オブジェクトの軌跡から時空間パターンを発見する。空間コロケーションマイニング [8] は空間的に共起する特徴を発見する。これらは空間的分布を扱うが、スライディングウィンドウによるサポートの時空間集計や密集領域の連結成分抽出は対象としない。

2.5 密部分テンソルマイニング

DenseAlert [10] や M-Zoom [9] は多次元テンソルにおける密ブロックを検出する。これらはテンソル全体のランク分解に基づく異常検出が主目的であり、アイテムセットパターンのスライディングウィンドウサポートや 6 近傍連結成分としての密集領域抽出は扱わない。

2.6 空間スキャン統計

Kulldorff [6] の空間スキャン統計は地理的領域における疾患クラスターを検出する代表的手法である。矩形窓を用いた時空間拡張 [7] も提案されているが、アイテムセットパターンのサポートではなく点過程の密度を対象とし、本研究とは問題設定が異なる。

Table 1: 関連手法との比較。✓: 対応、×: 非対応。

手法	アイテムセット	時空間密集領域	スライディングウィン
Apriori [1]	✓	×	×
LPFIM / LPPM / Recur	✓	×	✓
Apriori-Window [2]	✓	× (1D)	✓
空間スキャン統計 [6]	×	✓	×
DenseAlert [10]	×	✓	×
コロケーション [8]	✓	✓	×
本研究	✓	✓ (3D)	✓

2.7 本研究の位置づけ

表 1 に各手法との比較を示す。本研究は、Apriori-Window [2] の t 軸ストライドと Apriori 枝刈りを 3 次元接頭辞和・XY イベントリストと組み合わせることで、時空間アイテムセットの密集領域を効率的に発見する唯一のフレームワークである。

3 問題定義

本節では、まず Apriori-Window [2] が扱う 1 次元設定を整理し（3.1 節）、続いてそれを d 次元に拡張した本研究の問題定義を与える（3.2 節）。

3.1 Apriori-Window（1 次元）の復習

$\mathcal{I} = \{i_1, \ldots, i_m\}$ をアイテム集合、$\mathcal{D}_{1\mathrm{D}} = \{(t_n, d_n)\}_{n=1}^{N}$ をタイムスタンプ付きトランザクション列とする。$t_n \in \mathbb{Z}_{\geq 0}$ は整数タイムスタンプ（または離散化済みビン）、$d_n \subseteq \mathcal{I}$ はアイテム集合である。

パターン $P \subseteq \mathcal{I}$ の出現集合を $\mathrm{occ}(P) = \{t_n : P \subseteq d_n\}$ と定義する。

定義 1 (区間サポート [2]). ウィンドウ幅 W_t のもとで、ウィンドウ左端 l に対するパターン P の区間サポートを

$$\sup(P,\, l,\, W_t) = \left|\{t \in \mathrm{occ}(P) : l \leq t < l + W_t\}\right|$$

と定義する。

定義 2 (密集窓・密集区間 [2]). サポート閾値 θ に対し、$\sup(P, l, W_t) \geq \theta$ を満たすとき、窓 $[l, l+W_t]$ を P の密集窓と呼ぶ。密集窓の開始点 l が連続する最長区間 $[s, e]$ を P の密集区間と呼ぶ。

Apriori-Window [2] は、t 軸ストライドによる不要スキャンの排除と候補区間の交差に基づく Apriori 枝刈りにより、全パターンの密集区間を効率的に列挙する。

3.2 d 次元への拡張

本研究では、各トランザクションが時刻 c_0（時間軸）に加えて $d-1$ 個の追加座標 $(c_1, \ldots, c_{d-1})$（空間・カテゴリなど）を持つ d 次元トランザクションデータを対象とする。

記法. $\mathcal{D} = \{(\boldsymbol{c}_n, d_n)\}_{n=1}^{N}$ を d 次元トランザクション列とする。ここで $\boldsymbol{c}_n = (c_n^{(0)}, c_n^{(1)}, \ldots, c_n^{(d-1)}) \in \mathbb{R}^d$ は座標ベクトル（第 0 軸は時間）、$d_n \subseteq \mathcal{I}$ はアイテム集合である。

定義 3 (d 次元ビンインデックス). ビン幅ベクトル $\boldsymbol{\Delta} = (\Delta_0, \ldots, \Delta_{d-1})$ のもとで、トランザクション $(\boldsymbol{c}, d)$ のビンインデックスを

$$\boldsymbol{b}(\boldsymbol{c}) = \left(\lfloor c^{(k)} / \Delta_k \rfloor\right)_{k=0}^{d-1}$$

と定義する。グリッドサイズを $G = \prod_{k=0}^{d-1} N_k$ とする（N_k は第 k 次元のビン数）。

定義 4 (d 次元アイテム出現グリッド). パターン $P \subseteq \mathcal{I}$ の出現グリッド I_P を

$$I_P[\boldsymbol{b}] = \left|\{n : \boldsymbol{b}(\boldsymbol{c}_n) = \boldsymbol{b},\ P \subseteq d_n\}\right|$$

と定義する。すなわち、ビン $\boldsymbol{b}$ においてパターン P を含むトランザクション数である。

定義 5 (d 次元スライディングウィンドウサポート). ウィンドウビン幅ベクトル $\boldsymbol{W} = (W_0, W_1, \ldots, W_{d-1})$ のもとで、グリッド点 $\boldsymbol{s} = (s_0, s_1, \ldots, s_{d-1})$ を開始点とするウィンドウ内のパターン P のサポートを

$$S_P(\boldsymbol{s}) = \sum_{\boldsymbol{b} \in \prod_{k=0}^{d-1} [s_k, s_k + W_k)} I_P[\boldsymbol{b}] \tag{1}$$

と定義する。全グリッド点 $\boldsymbol{s}$ にわたる $S_P(\boldsymbol{s})$ を P のサポートサーフェスと呼ぶ。

注意 1 (1 次元設定への帰着). $d = 1$（時間軸のみ）のとき、$S_P(s_0) = \sup(P, s_0, W_0)$（定義 *1*）に帰着する。すなわち本研究の問題設定は *Apriori-Window [2]* の真の拡張である。さらに $d = 3$（時間 × 空間 x× 空間 y）のとき、本稿で示す *3* 次元実験結果が得られる。

定義 6 (密集セル). サポート閾値 θ に対し、グリッド点 $\boldsymbol{s}$ が $S_P(\boldsymbol{s}) \geq \theta$ を満たすとき、$\boldsymbol{s}$ を密集セルと呼ぶ。パターン P の密集セル集合を $\mathcal{L}_\theta(S_P)$ と表す。

定義 7 (密集領域). 密集セル集合 $\mathcal{L}_\theta(S_P)$ の $2d$-近傍連結成分（各次元の正負方向 $\pm\boldsymbol{e}_k$ を隣接と定義）の各成分を密集領域と呼ぶ。$d = 3$ のとき $2d = 6$ 近傍（辺共有隣接）となる。

1 次元の「密集区間」（定義 2）は、$d = 1$ のときの密集領域に対応する。

Apriori 包含定理.

補題 1 (密集領域の単調性). 任意のパターン P とその部分集合 $Q \subset P$ に対して、$\mathcal{L}_\theta(S_P) \subseteq \mathcal{L}_\theta(S_Q)$ が成立する。

Proof. 任意のビン $\boldsymbol{b}$ に対し、$P \subseteq d_n$ ならば $Q \subseteq d_n$ であるから $I_P[\boldsymbol{b}] \leq I_Q[\boldsymbol{b}]$。接頭辞和の単調性より $S_P(\boldsymbol{s}) \leq S_Q(\boldsymbol{s})$。 □

補題 1 は Apriori-Window の候補区間交差枝刈り [2] の d 次元版であり、$k \geq 2$ の枝刈りの根拠である（4.3 節）。

3.3 d 次元密集領域検出問題

問題 1 (d 次元密集領域検出). **入力**：d 次元トランザクション列 $\mathcal{D}$、ビン幅 $\boldsymbol{\Delta} \in \mathbb{R}^d$、ウィンドウビン数 $\boldsymbol{W} \in \mathbb{Z}^d_{>0}$、サポート閾値 θ、最大パターン長 $L_{\max}$。
出力：各パターン P に対し、P の密集領域の全集合 $\{\mathcal{R}_1, \mathcal{R}_2, \ldots\}$（各 $\mathcal{R}_j$ は密集セルの連結成分）。

問題 1 の計算量はサポートサーフェス S_P の計算コストに支配される。単純実装の $O\left(M \cdot G \cdot \prod_{k=0}^{d-1} W_k\right)$ を提案アルゴリズム（4 節）では $O\left(M \cdot G + N \cdot \prod_{k=1}^{d-1} W_k\right)$ に削減する（第 0 軸を時間軸 t とする）。

Table 2: Apriori-Window [2] と本研究の技術対応表。

Apriori-Window（1 次元）	本研究（d 次元）
累積サム $O(1)$ 区間クエリ	d 次元接頭辞和 $O(2^d)$ ウィンドウクエリ
t 軸ストライド（$k = 1$）	空間イベントリスト + t 軸ストライド（$k = 1$）
候補区間交差による枝刈り	候補セル集合交差による枝刈り（$k \geq 2$）
密集区間（1 次元連結成分）	密集領域（$2d$-近傍連結成分）

4 提案手法: d 次元 APRIORI-WINDOW アルゴリズム

問題 1 を解くための **d 次元 Apriori-Window アルゴリズム**を提案する。以下の 3 つの高速化技術を組み合わせ（Algorithm 2）、計算量を $O\left(M \cdot G + N \cdot \prod_{k=1}^{d-1} W_k\right)$ に抑える（第 0 軸を時間軸 t とする）。各技術は Apriori-Window [2] の対応する 1 次元技術の直接拡張である（表 2）。

4.1 d 次元接頭辞和による $O(2^d)$ ウィンドウクエリ

出現グリッド I_P（サイズ $\prod N_k$）に対し、各次元を +1 パディングした d 次元接頭辞和 PS_P（サイズ $\prod(N_k + 1)$）を構築する：

$$\mathrm{PS}_P[\boldsymbol{b}] = \sum_{\boldsymbol{b}' \prec \boldsymbol{b}} I_P[\boldsymbol{b}'], \tag{2}$$

ここで $\boldsymbol{b}' \prec \boldsymbol{b}$ は成分ごとに $b'_k < b_k$（全次元）を意味する。PS_P は各次元を順番に累積和をとることで $O(G)$ で構築できる。構築後、任意のウィンドウサポートは 2^d コーナーの包除原理により

$$S_P(\boldsymbol{s}) = \sum_{\boldsymbol{c} \in \{0,1\}^d} (-1)^{d - |\boldsymbol{c}|_1} \mathrm{PS}_P[\boldsymbol{s} + \boldsymbol{c} \odot \boldsymbol{W}] \tag{3}$$

で $O(2^d)$ に取得できる（$\boldsymbol{c} \odot \boldsymbol{W}$ は成分積）。$k \geq 2$ ではこれを全グリッド点に適用し、サポートサーフェス S_P を $O(2^d \cdot G)$ で計算する。実用的な d（≤ 5 程度）では 2^d は定数とみなせる。

4.2 t 軸 1 次元ストライド最適化（$k = 1$）

$k = 1$（単体アイテム）の密集セル検出では、Apriori-Window の t 軸ストライド [2] を各空間ウィンドウ (x_s, y_s) に独立に適用する。

空間イベントリストの構築. トランザクション $(\boldsymbol{c}_n, d_n)$（ビン化後: $\boldsymbol{b}_n$）がアイテム i を含む場合、i が関係する空間ウィンドウ開始点の第 k 成分（$k \geq 1$）の範囲は

$$s_k \in [\max(0,\ b_n^{(k)} - W_k + 1),\ b_n^{(k)}]$$

であるから、全空間次元の組み合わせ $(s_1, \ldots, s_{d-1})$ に $b_n^{(0)}$（時間ビン）を追加する。構築コストは $O\left(N_i \cdot \prod_{k=1}^{d-1} W_k\right)$。$d = 1$（時間のみ）のとき単一リスト $L_i() = \mathrm{occ}(\{i\})$ に帰着する。

t 軸ストライド. 各 (x_s, y_s) について、ソート済みリスト $L_i(x_s, y_s)$ に対しウィンドウ左端 l を動かして密集窓の開始点を求める（Algorithm 1）。本アルゴリズムは Apriori-Window の t 軸ストライドと同一のジャンプ規則を使用する。
ウィンドウ $[l, l + W_t)$ 内のカウント c が θ 未満の場合、次のイベントを含む最小の窓開始点へジャンプする：

$$l \leftarrow \max(l + 1,\ L_i(x_s, y_s)[\mathrm{next}] - W_t + 1). \tag{4}$$

Algorithm 1 t 軸 1 次元ストライドによる密集ウィンドウ開始点列挙

Require: ソート済みイベントリスト L、ウィンドウ幅 W_t、閾値 θ
Ensure: 密集ウィンドウ開始点の集合 $\mathcal{S}$

$\mathcal{S} \leftarrow \emptyset$, $l \leftarrow L[0] - (W_t - 1)$, $prev \leftarrow$ nil
while $l \le L[\text{last}]$ **do**
  $l_{\text{lo}} \leftarrow \text{lower_bound}(L, l)$, $l_{\text{hi}} \leftarrow \text{upper_bound}(L, l + W_t - 1)$
  $c \leftarrow l_{\text{hi}} - l_{\text{lo}}$
  if $c \ge \theta$ **then**
    if $prev \neq$ nil **then**
      $\mathcal{S} \leftarrow \mathcal{S} \cup \{prev + 1, \dots, l - 1\}$ ▷ ストライドで飛ばした区間を補填
    end if
    $\mathcal{S} \leftarrow \mathcal{S} \cup \{l\}$, $prev \leftarrow l$
    if $c > \theta$ **then**
      $l \leftarrow \max(l + 1, L[l_{\text{lo}} + (c - \theta)])$ ▷ 式 (5)
    else
      $l \leftarrow l + 1$
    end if
  else
    $prev \leftarrow$ nil
    $l \leftarrow \max(l + 1, L[l_{\text{hi}}] - W_t + 1)$ ▷ 式 (4)；$L[l_{\text{hi}}]$ が次イベント
  end if
end whilereturn $\mathcal{S}$

カウント $c > \theta$ の場合（余剰 surplus $= c - \theta$ 個）、左端から surplus 番目のイベントの時刻へジャンプする：

$$l \leftarrow L_i(x_s, y_s)[l_{\text{lo}} + \text{surplus}]. \tag{5}$$

l_{lo} はリスト中で l 以上の最初のインデックスである。

補題 2 (ストライドの正確性). 式 (5) のジャンプ先 l' に対し、全ての $l'' \in (l, l')$ についても $c(l'') \ge \theta$ が成立する。すなわち、スキップされた位置も全て密集窓の開始点である。

Proof. $l' = L_i[l_{\text{lo}} + \text{surplus}]$ とおく。任意の $l'' \in (l, l')$ に対し、ウィンドウ左端を l から l'' に進めると $L_i[l_{\text{lo}}], \dots, L_i[l_{\text{lo}} + \text{surplus} - 1]$ のうち l'' 未満の要素がウィンドウから脱出するが、$L_i[l_{\text{lo}} + \text{surplus}] = l' > l''$ より $L_i[l_{\text{lo}} + \text{surplus}]$ はまだウィンドウ内に残る。したがって $c(l'') \ge c(l) - \text{surplus} = \theta$。 □

補題 2 は Apriori-Window の命題 3 [2] の d 次元版であり、スキップ区間内の密集ウィンドウ開始点を陽に評価せずとも補填により正確に列挙できることを示す。

4.3 Apriori 枝刈りによる候補セル限定（$k \ge 2$）

$k \ge 2$ のパターン $P = \{i_1, \dots, i_k\}$ について、補題 1 より

$$\mathcal{L}_\theta(S_P) \subseteq \bigcap_{Q \subset P, |Q| = k-1} \mathcal{L}_\theta(S_Q). \tag{6}$$

右辺の交差集合を候補セル集合 $C(P)$ と定義する。これは Apriori-Window の「候補区間交差」[2] の d 次元版であり、密集セル集合の交差として定式化される。$C(P) = \emptyset$ ならば P の密集セルは存在しないと確定し、I_P の構築・接頭辞和の計算を省略できる。

Algorithm 2 d 次元 Apriori-Window アルゴリズム

Require: d 次元トランザクション $\mathcal{D}$、パラメータ $(\boldsymbol{W}, \theta, L_{\max})$
Ensure: 各パターン P の密集領域集合 $\{\mathcal{R}_P\}$

離散化: $\mathcal{D}$ を (b_n, d_n) に変換、グリッドサイズ $(N_0, \dots, N_{d-1})$ を決定
for all $i \in \mathcal{I}$ **in parallel do** ▷ $k = 1$; Rayon 並列
  $L_i(s_{1:}) \leftarrow$ 空間イベントリスト構築（§4.2）
  for all $s_{1:} \in \prod_{k=1}^{d-1} [0, N_k)$ **do**
    if $L_i(s_{1:}) \neq \emptyset$ **then**
      $\mathcal{S} \leftarrow \text{TStrideStarts}(L_i(s_{1:}), W_0, \theta)$ ▷ Algorithm 1
      各 $s_0 \in \mathcal{S}$ を密集セル集合 D_i に追加
    end if
  end for
  $\mathcal{R}_{\{i\}} \leftarrow 2d$-近傍 Union-Find 連結成分 of D_i
end for
$\mathcal{F}_1 \leftarrow \{\{i\} : \mathcal{R}_{\{i\}} \neq \emptyset\}$
while $\mathcal{F}_{k-1} \neq \emptyset$ **and** $k \le L_{\max}$ **do** ▷ $k \ge 2$
  cands $\leftarrow$ GenerateCandidates($\mathcal{F}_{k-1}$)
  cands $\leftarrow$ PruneCandidates(cands, $\mathcal{F}_{k-1}$)
  for all $P \in$ cands **in parallel do**
    $C(P) \leftarrow \bigcap_{Q \subset P, |Q| = k-1} \mathcal{L}_\theta(S_Q)$ ▷ 式 (6)
    if $C(P) = \emptyset$ **then continue**
    end if
    $I_P \leftarrow \text{BuildItemGrid}(\mathcal{D}, P, C(P))$
    $\text{PS}_P \leftarrow \text{BuildPrefixSumND}(I_P)$
    $S_P \leftarrow \text{ComputeSupportSurface}(\text{PS}_P, \boldsymbol{W})$
    $\mathcal{R}_P \leftarrow 2d$-近傍 Union-Find 連結成分 of $\{\boldsymbol{s} : S_P(\boldsymbol{s}) \ge \theta\}$
  end for
  $\mathcal{F}_k \leftarrow \{P : \mathcal{R}_P \neq \emptyset\}$, $k \leftarrow k + 1$
end whilereturn $\{R_P : P \in \bigcup_k \mathcal{F}_k\}$

Table 3: d 次元 Apriori-Window 各ステップの計算量。N: トランザクション数、$G = \prod_{k=0}^{d-1} N_k$、$W_{\text{sp}} = \prod_{k=1}^{d-1} W_k$（空間ウィンドウ積）、$C_k$: k 段候補数。

ステップ	計算量	備考
離散化	$O(N)$	
$k = 1$: 空間イベントリスト構築	$O(N \cdot W_{\text{sp}})$	全アイテム合計
$k = 1$: t 軸ストライド	$O(\lvert\mathcal{S}_i\rvert + \log N_i)$ per 空間キー	$\lvert\mathcal{S}_i\rvert$: 密集点数
$k \ge 2$: 接頭辞和構築	$O(2^d \cdot G)$ per 候補	Apriori 枝刈り後
$k \ge 2$: I_P 構築	$O(N)$ per 候補	
連結成分	$O(\lvert\mathcal{L}\rvert \cdot \alpha)$ per パターン	α: 逆アッカーマン

$C(P) \neq \emptyset$ の場合は、$C(P)$ 内のトランザクションのみを走査して I_P を構築し（$O(N)$）、接頭辞和 PS_P（$O(G)$）・サポート曲面（$O(G)$）を計算する。

4.4 アルゴリズム全体と計算量

支配的コストは $k = 1$ の空間イベントリスト構築 $O(NW_{\text{sp}})$ と、$k \ge 2$ の接頭辞和構築 $O(C_k \cdot 2^d \cdot G)$（C_k: Apriori 枝刈り後の候補数）である。ナイーブ実装の $O\left(M \cdot G \cdot \prod_{k=0}^{d-1} W_k\right)$ に対して、t 軸ストライドはウィンドウ幅方向の削減、Apriori 枝刈りはパターン次元の削減を担う。次元数 d が増えると 2^d と G の双方が

Table 4: EX-A: 3 次元設定 vs 1 次元相当設定の比較（5 シード、$k = 2$ パターン）。

設定	検出 $k \geq 2$ パターン数	植え付け回収数	Precision	Recall
3 次元（W_x=W_y=3）	3	3	1.00	1.00
1 次元相当（W_x=W_y=50）	≈93–100	3	≈0.03	1.00

増加するが、実用的な範囲（$d \leq 5$）では管理可能である（EX-E; 5.6 節）。

5 実験

以下の研究課題を検証する。**RQ1**: 1 次元設定（空間次元なし）と比べて 3 次元設定は空間的偽陽性を抑制できるか？ **RQ2**: データサイズ N に対して計算時間はどうスケールするか？ **RQ3**: 各高速化技術はどの程度の効果をもたらすか？（アブレーション）**RQ4**: 既存手法（FIM、1 次元 AW、Span_3D、MZoom_3D）と比べてどの程度優れているか？ **RQ5**: 次元数 d の増加に対して計算時間はどうスケールするか？

5.1 実験環境

すべての実験は Rust（release build、Rayon 並列化）で実装した `md_pipeline` バイナリを用い、Apple M3（8 コア、24 GB RAM）上で実行した。デフォルトパラメータ：$\theta = 3$（サポート閾値）、$W_t = 5$（時間ウィンドウ）、$W_x = W_y = 3$（空間ウィンドウ）、$L_{\max} = 3$（最大パターン長）、$\Delta t = \Delta x = \Delta y = 1.0$。

5.2 EX-A: 3 次元 vs 1 次元相当の比較

目的. Apriori-Window [2] は時間軸のみを扱うため、空間的に局在するパターンを識別できない（全空間を集約する）。本実験では合成データを用いて、3 次元設定が空間的偽陽性を抑制することを実証する。

合成データ設計. $N = 5{,}000$ トランザクション、アイテム数 $|\mathcal{I}| = 10$、空間範囲 $[0, 50]^2$、時刻範囲 $[0, 100]$ の合成データを生成する。3 つの密集領域を植え付ける：各領域は 2 アイテムパターン $P_1 = \{1, 2\}$、$P_2 = \{3, 4\}$、$P_3 = \{5, 6\}$ を割り当て、それぞれランダムな時空間中心 (t_c, x_c, y_c) の近傍に 20 件の集中トランザクションを注入する。残りの $N - 60$ 件はバックグラウンドノイズ（ランダム）。

比較設定.

- **3 次元設定**（本提案）：$W_x = W_y = 3$（植え付け領域に対応する局所ウィンドウ）
- **1 次元相当設定**：$W_x = W_y = 50$（空間範囲全体をカバーする大ウィンドウで空間次元を事実上無視し、1 次元集約と同等の挙動を実現）

どちらも $\theta = 3$、$W_t = 5$ を使用する。

評価指標. 検出された $k = 2$ パターンの集合と植え付けパターン集合の一致で Precision・Recall を計算する。5 シードで実験する。

結果. 表 4 に結果を示す。

分析. (RQ1) 3 次元設定では全 5 シードで植え付け 3 パターン $\{1, 2\}$、$\{3, 4\}$、$\{5, 6\}$ を完全回収し、偽陽性パターンは 0 件であった（Precision = 1.0、Recall = 1.0）。一方、1 次元相当設定では $\{1, 3\}$、$\{1, 4\}$、$\{2, 3\}$ などのバックグラウンドノイズ由来の偽陽性パターンが多数（93〜100 件）検出された（Precision ≈ 0.03）。

これは、空間次元を集約すると空間的に異なる位置のトランザクションが同一ウィンドウに混入し、偶発的な共起が閾値を超えることを示す。空間次元の明示的なモデル化が精度向上に本質的に重要である。

Table 5: EX-B: N に対するスケーラビリティ（W_x=W_y=3、W_t=5）。

N	検出パターン数	実行時間 (s)
1,000	9	0.005
5,000	13	0.021
10,000	13	0.030
50,000	55	0.455
100,000	59	1.407

Table 6: EX-B: 空間ウィンドウサイズに対するスケーラビリティ（N=20,000、W_t=5）。

$W_x = W_y$	検出パターン数	実行時間 (s)
1	13	0.010
2	13	0.034
3	22	0.086
5	55	0.477
7	55	1.150
10	86	1.991

5.3 EX-B: データサイズに対するスケーラビリティ

実験設計. $N \in \{1{,}000,\ 5{,}000,\ 10{,}000,\ 50{,}000,\ 100{,}000\}$ とウィンドウサイズ $W_x = W_y \in \{1, 2, 3, 5, 7, 10\}$（$N = 20{,}000$ 固定）を変化させ、実行時間を計測する。

結果. 表 5（N 方向）と表 6（ウィンドウ方向）に結果を示す。

分析. (RQ2) 表 5 より、実行時間は N に対してほぼ線形にスケールする。$N = 100{,}000$ でも約 1.5 秒を達成しており、大規模データでの実用性を示す。

表 6 より、空間ウィンドウサイズの増大に対しては超線形のスケーリングが観察される。$W_x = W_y = 1$ から $W_x = W_y = 3$ まではほぼ W^2 に比例するが（0.010s → 0.086s、8.6×）、$W_x = W_y = 7$ 以上では XY イベントリスト構築の $O(NW_xW_y)$ コストが支配的となる。この結果は計算量分析（表 3）と整合的である。

5.4 EX-C: 高速化技術のアブレーション

実験設計. Apriori 枝刈りの効果を測定するため、$L_{\max}$ を 1 から 2、3、4 と増やしながら実行時間を計測する。真の頻出パターンが $k = 2$ で打ち止まる合成データ（$N = 20{,}000$）を使用する。

結果と分析. (RQ3) $L_{\max}$ を 1 から 2、3、4 と増やしても実行時間はほぼ不変（0.062 → 0.083 → 0.082 → 0.087 s）であった。これは Apriori 枝刈りが $k \geq 3$ の候補を完全に排除していることを示す。当該データでは $k = 3$ を満たすパターンが存在せず、$C(P) = \emptyset$ の枝刈りにより I_P 構築が省略されたためである。この

Table 7: EX-D: グリッドサーチのパラメータ範囲。

手法	探索パラメータ
FIM_global	$\theta \in \{3, 5, 7, 10, 15, 20, 30\}$
1D_AW	$W_t \in \{3, 5, 7, 10\}$、$\theta \in \{3, 5, 7, 10\}$
Span_3D	$\theta \in \{3, 5, 7, 10, 15\}$
MZoom_3D	$\theta \in \{3, 5, 7\}$、最小密度 $\in \{0.1, 0.3, 0.5\}$
3 次元 AW	$W_t \in \{3, 5, 7, 10\}$、W_x=$W_y \in \{2, 3, 5, 7\}$、$\theta \in \{3, 5, 7, 10\}$

Table 8: EX-D: 比較実験結果（合成データ、5 シード平均 ± 標準偏差）。植え付け $k = 2$ パターン 3 件の回収率で評価。

手法	Precision	Recall	F1	検出パターン数
FIM_global	0.016 ± 0.000	**1.000** ± 0.000	0.031 ± 0.000	190.0
1D_AW	0.950 ± 0.100	**1.000** ± 0.000	0.971 ± 0.057	3.8
Span_3D	**1.000** ± 0.000	0.733 ± 0.133	0.840 ± 0.080	2.2
MZoom_3D	**1.000** ± 0.000	**1.000** ± 0.000	**1.000** ± 0.000	3.0
3 次元 AW（本提案）	**1.000** ± 0.000	**1.000** ± 0.000	**1.000** ± 0.000	3.0

挙動は Apriori-Window [2] の 1 次元枝刈り効果と同様であり、3 次元への拡張でも同じ恩恵が得られることを示す。

5.5 EX-D: 比較手法との性能比較

目的. 本実験では 5 種類の手法を合成データ上で比較し、3 次元 Apriori-Window の有効性を検証する（RQ4）。

比較手法.

- **FIM_global**: グローバル頻出アイテムセットマイニング [1]。空間・時間構造を無視し、全トランザクション上の共起頻度のみを使用する。
- **1D_AW**: Apriori-Window [2]（1 次元、時間軸のみ）。空間次元を集約し、時刻スライディングウィンドウでサポートを計算する。
- **Span_3D**: 空間的点密度手法。($1 \times 1 \times 1$) セルごとの共起頻度が閾値を超えるパターンを検出する。
- **MZoom_3D**: 密部分テンソル検出 [9] の 3 次元近似。時空間グリッドブロック内の最大密度に基づいてパターンを検出する。
- **3 次元 AW**（本提案）: 3 次元スライディングウィンドウサポートを用いた提案手法。

実験設計. N = 5,000 トランザクション（アイテム数 $|\mathcal{I}| = 20$、空間範囲 $[0, 50]^2$、時刻範囲 $[0, 100]$）の合成データを生成し、3 つの密集領域を植え付ける（各領域 $P_1 = \{1, 2\}$、$P_2 = \{3, 4\}$、$P_3 = \{5, 6\}$、各 20 件注入、ジッタ：$t \pm 2.0$、$x \pm 1.0$、$y \pm 1.0$）。バックグラウンドノイズは全アイテム（1–20）のランダム組み合わせ。

全手法でグリッドサーチを実施し（パラメータ範囲を表 7 に示す）、F1 を最大化するパラメータを選択する。5 シードで実験し平均を報告する。

結果. 表 8 に各手法の最良パラメータにおける検出性能を示す。

分析. (RQ4) **FIM_global** は全 3 パターンを回収（Recall = 1.0）するが、190 件の偽陽性パターンを生成する（Precision = 0.016）。これは空間・時間構造を無視することで、バックグラウンドノイズ由来の偶発的共起が全てのパターンとして報告されるためである。

1D_AW（Apriori-Window [2]）は時間軸のみを使用し、大半のシードで完全回収を達成する（Recall = 1.0、F1 平均 = 0.971）が、1 シード（seed=456）で偽陽性パターン 1 件が発生した（検出数 4 件）。これは時刻が近接する異なる空間位置のトランザクションが同一時間ウィンドウに混入するためである。

Span_3D は高精度（Precision = 1.0）だが、平均 Recall = 0.733（3 シードで 2 件回収、2 シードで 3 件回収）。ウィンドウ密度ではなく点密度を使用するため、注入領域（ジッタ幅 ±2 の空間的広がり）の共起が分散した場合に閾値未達となる。

MZoom_3D（近似実装）はブロック密度によるパターン検出であり、本実験では F1 = 1.0 を達成した。ただし MZoom の本来の目的はテンソル異常検出であり、アイテムセットのスライディングウィンドウサポート意味論は持たない。

3 次元 AW（本提案）は全シードで Precision = 1.0、Recall = 1.0 を達成し、唯一の手法として常に正確な 3 パターン検出を実現した。時空間スライディングウィンドウサポートにより、空間的に局在した密集アイテムセット領域を偽陽性なく発見できる。

5.6 EX-E: 次元数スケーラビリティ

目的. 次元数 d の増加が計算時間に与える影響を定量化する（RQ5）。

実験設計. N = 10,000 トランザクション固定、$d \in \{2, 3, 4, 5\}$ を変化させる。各次元のビンサイズ $\Delta_k = 1.0$、ウィンドウサイズ $W_k = 3$（全次元統一）、閾値 $\theta = 3$、$L_{\max} = 2$。座標範囲は各次元 $[0, 50]$ に統一し、密集領域は 3 件植え付ける（各次元の中心・ジッタを適宜設定）。5 シードの平均実行時間を計測する。

Table 9: EX-E: 次元数 d に対するスケーラビリティ（N=10,000、W_k=3、θ=3、5 シード平均）。

d	グリッドサイズ G	実行時間 (s)	備考
2	50×50	*TBD*	1D-AW 相当
3	$50 \times 50 \times 50$	*TBD*	本稿 EX-B 相当
4	50^4	*TBD*	
5	50^5	*TBD*	

結果. 表 9 に結果を示す。

分析. (RQ5) *[実装完了後に実測値で更新]*

5.7 考察

密集領域の数と空間ウィンドウ. $W_x = W_y$ の増大に伴い検出パターン数が増加する（表 6）。EX-A の結果と合わせると、空間ウィンドウを大きくするほど偽陽性パターンが増加することがわかる。適切な空間ウィンドウサイズはデータの空間粒度に依存し、応用ドメインの知識に基づいた設定が重要である。

6 結論

本研究では、Apriori-Window [2] を 1 次元（時間軸）から 3 次元（時間 × 空間 x × 空間 y）に拡張した **3 次元 Apriori-Window アルゴリズム**を提案した。3 つの高速化技術—3 次元接頭辞和、t 軸ストライド最適化、Apriori 枝刈り—はいずれも Apriori-Window

の対応する 1 次元技術の直接拡張であり、ナイーブ実装の $O(M \cdot G \cdot W_t W_x W_y)$ から $O(M \cdot G + N \cdot W_x W_y)$ へ計算量を削減した。

合成データ実験では以下を確認した：(1) **EX-A**: 3 次元設定（$W_x = W_y = 3$）は植え付け 3 パターンを Precision = 1.0、Recall = 1.0 で完全回収したのに対し、1 次元相当設定（$W_x = W_y = 50$）は背景ノイズ由来の偽陽性パターンを多数検出した（Precision ≈ 0.03）。空間次元の明示的なモデル化が精度に本質的に重要であることを示す。(2) **EX-B**: $N = 100{,}000$ トランザクションで約 1.4 秒を達成し、N に対してほぼ線形のスケーリングを示した。(3) **EX-C**: Apriori 枝刈りにより $k \geq 3$ の不要な計算が完全に排除された。(4) **EX-D**: 比較実験（5 手法、5 シード）では、3 次元 AW のみが Precision = 1.0、Recall = 1.0（F1 = 1.0）を全シードで達成した。FIM_global（Precision = 0.016）は空間局在化不可、1D_AW（F1 平均 = 0.971）は時刻近接による偽陽性が発生、Span_3D（F1 平均 = 0.840）は点密度の厳格さによる見逃しが生じた。

制約と限界. (1) 空間次元はグリッド離散化を前提とし、ビン幅の選択が結果に影響する。(2) 空間ウィンドウサイズが大きい場合（$W_x W_y \gtrsim 100$）、XY イベントリスト構築コストが支配的となり実行時間が増大する。(3) 本手法は密集領域の検出に特化しており、検出された密集領域と外部イベントとの統計的関連付けは行わない。

今後の課題. (1) 実データ（小売 POS データ、疫学データ等）への応用と評価。(2) 検出された密集領域を外部イベントに帰属させるパイプライン（Apriori-Window [2] の後続研究 [3]）への接続。(3) ストリーミング設定へのオンライン更新対応。(4) 適応的グリッド解像度選択。

ACKNOWLEDGMENT

This work is supported by the Honda Bridge Project.

REFERENCES

[1] Rakesh Agrawal and Ramakrishnan Srikant. 1994. Fast algorithms for mining association rules. In *Proceedings of VLDB*. 487–499.
[2] Anonymous. 2025. スライディング窓による区間局所頻度の直接評価を用いた密集パターンの厳密抽出. In *Proceedings of DSAA*. Under review.
[3] Anonymous. 2025. 頻出パターンの変動はなぜ起きたか？ サポート時系列と外部イベントの統計的帰属パイプライン. In *Proceedings of DSAA*. Under review.
[4] Fosca Giannotti, Mirco Nanni, Fabio Pinelli, and Dino Pedreschi. 2007. Trajectory pattern mining. In *Proceedings of KDD*. 330–339.
[5] Jiawei Han, Jian Pei, and Yiwen Yin. 2000. Mining frequent patterns without candidate generation. *ACM SIGMOD Record* 29, 2 (2000), 1–12.
[6] Martin Kulldorff. 1997. A spatial scan statistic. *Communications in Statistics-Theory and Methods* 26, 6 (1997), 1481–1496.
[7] Martin Kulldorff. 2001. Prospective time periodic geographical disease surveillance using a scan statistic. *Journal of the Royal Statistical Society: Series A* 164, 1 (2001), 61–72.
[8] Shashi Shekhar and Yan Huang. 2001. Discovering spatial co-location patterns: A summary of results. *Advances in Spatial and Temporal Databases* (2001), 236–256.
[9] Kijung Shin, Bryan Hooi, and Christos Faloutsos. 2018. Fast, accurate, and flexible algorithms for dense subtensor mining. *ACM Transactions on Knowledge Discovery from Data* 12, 3 (2018), 1–30.
[10] Kijung Shin, Bryan Hooi, Jisu Kim, and Christos Faloutsos. 2017. DenseAlert: Incremental dense-subtensor detection in tensor streams. In *Proceedings of KDD*. 1057–1066.